Cooperative Kernel Regression

Distributed Optimization, Federated Learning, and Differential Privacy

Agathe PASCAL Nastassia BONETTI

March 30, 2026

Abstract

This report presents a course-consistent implementation of Gaussian-kernel ridge regression with a Nyström approximation. The experiments cover distributed optimization over a communication graph, federated learning, and a differentially private distributed variant. The focus is on the mathematical formulation, stable parameter choices, convergence plots, and a concise interpretation of the observed behavior of each method.

Project repository: https://github.com/pgatt06/Coop_optim.git

1 Common formulation

The kernel is

$$k(x, \xi) = \exp\left(-(x - \xi)^2\right).$$

Given n samples (x_i, y_i) and m Nyström landmarks $\tilde{x}_1, \ldots, \tilde{x}_m$, define

$$(K_{mm})_{pq} = k(\tilde{x}_p, \tilde{x}_q), \qquad (K_{nm})_{ip} = k(x_i, \tilde{x}_p).$$

The centralized problem is

$$\alpha^\star \in \arg\min_{\alpha \in \mathbb{R}^m} \frac{\sigma^2}{2} \alpha^\top K_{mm} \alpha \; + \; \frac{1}{2} \|y - K_{nm}\alpha\|^2 \; + \; \frac{\nu}{2} \|\alpha\|^2,$$

with $\sigma = 0.5$ and $\nu = 1$. Since the objective is quadratic and strongly convex,

$$\alpha^\star = \left(K_{nm}^\top K_{nm} + \sigma^2 K_{mm} + \nu I_m\right)^{-1} K_{nm}^\top y.$$

This solution is used as the reference point throughout the experiments.

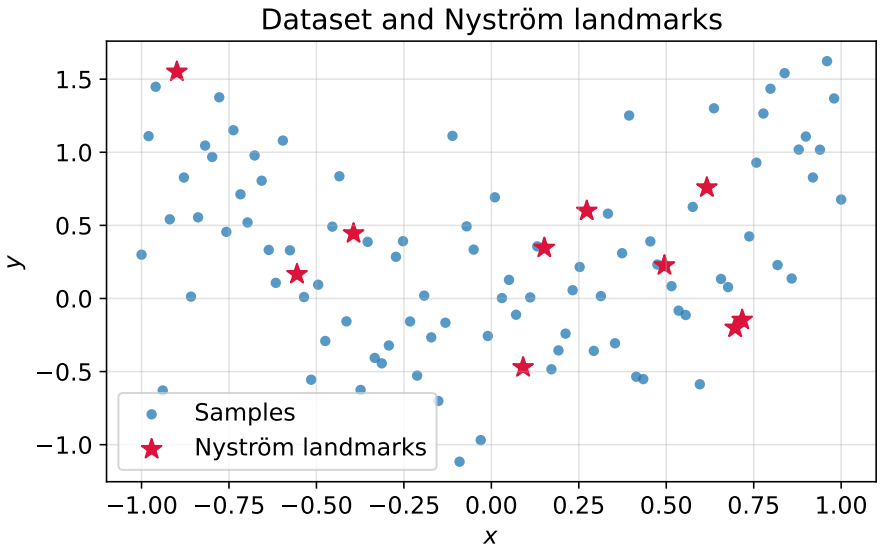

Figure 1: Samples used in Parts I and III, together with the selected $m = 10$ Nyström landmarks.

For the distributed setting, the first $n = 100$ points are split over $a = 5$ agents, hence 20 points per agent. With local data (K_i, y_i) at agent i,

$$f_i(\alpha) = \frac{\sigma^2}{2a}\alpha^\top K_{mm}\alpha \; + \; \frac{1}{2}\left\|y_i - K_i\alpha\right\|^2 \; + \; \frac{\nu}{2a}\left\|\alpha\right\|^2, \qquad \sum_{i=1}^{a} f_i(\alpha) = F(\alpha).$$

We monitor

$$g_i^t = \left\|\alpha_i^t - \alpha^\star\right\|, \qquad \bar{g}^t = \left\|\bar{\alpha}^t - \alpha^\star\right\|, \qquad \bar{\alpha}^t = \frac{1}{a}\sum_{i=1}^{a}\alpha_i^t.$$

2 Distributed optimization

Four graphs are tested: cycle, line, small-world, and complete. The undirected mixing matrices are built with Metropolis weights, so the standard course assumptions hold: symmetry, stochasticity, and connectivity.

The methods are DGD, Gradient Tracking (GT), dual decomposition, and ADMM. Their parameters are chosen from the local objective constants

$$L_{\max} = \max_i \lambda_{\max}(H_i), \qquad \mu_{\min} = \min_i \lambda_{\min}(H_i),$$

the graph mixing factor $\beta = \|W - J\|_2$, and for the dual method

$$L_d = \left\|B^\top H^{-1} B\right\|_2.$$

- **DGD:** $\eta_{\mathrm{DGD}} = \min\Big(0.9\frac{1-\beta}{L_{\max}},\, 0.9\frac{1}{L_{\max}}\Big)$. This keeps the method in the stable $O((1-\beta)/L)$ regime. With a constant step size, DGD converges linearly to a neighborhood of $\alpha^\star$ because the consensus error does not vanish exactly.
- **Gradient Tracking:** $\eta_{\mathrm{GT}} = 0.2/L_{\max}$. The tracking variable reconstructs the global average gradient; under smoothness and strong convexity, a small enough $O(1/L)$ step yields exact linear convergence.
- **Dual decomposition:** $\tau_{\mathrm{dual}} = 1/(2L_d)$. The dual objective is smooth and concave, so this is a conservative stable ascent step that leads here to the exact solution.
- **ADMM:** $\rho_{\mathrm{ADMM}} = \sqrt{\mu_{\min} L_{\max}}$. In quadratic consensus problems, any $\rho > 0$ gives the exact solution; this choice balances the primal and dual scales.

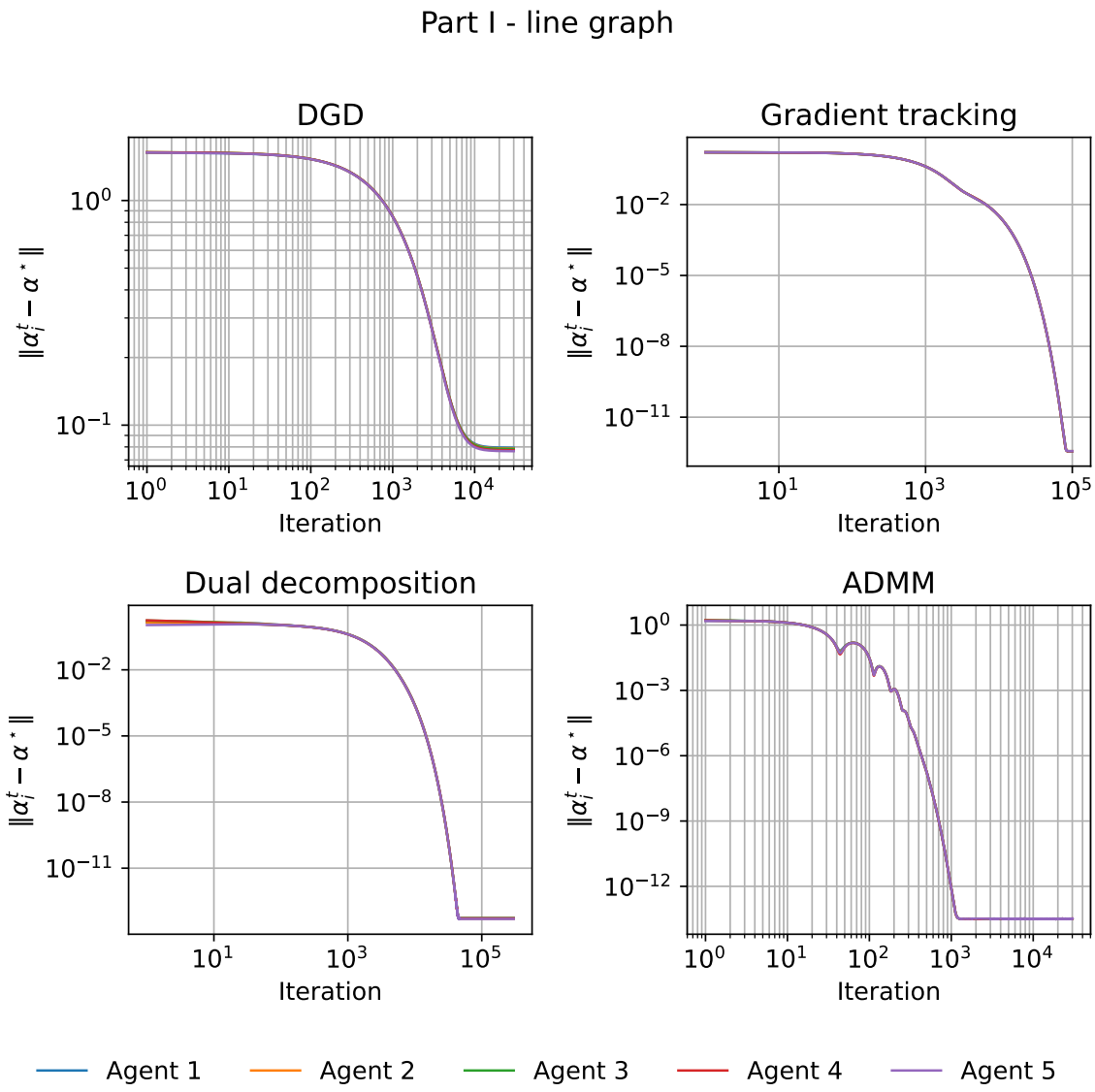

Figure 2: On the line graph, DGD converges to a neighborhood of the solution, whereas GT, dual decomposition, and ADMM reach numerical precision.

The line graph is representative: the worst final DGD gap is 7.93×10^{-2}, while the three other methods end below 4×10^{-13}. This matches the theory exactly: DGD with a constant step retains a consensus bias, whereas GT, the dual method, and ADMM are exact methods in this quadratic setting.

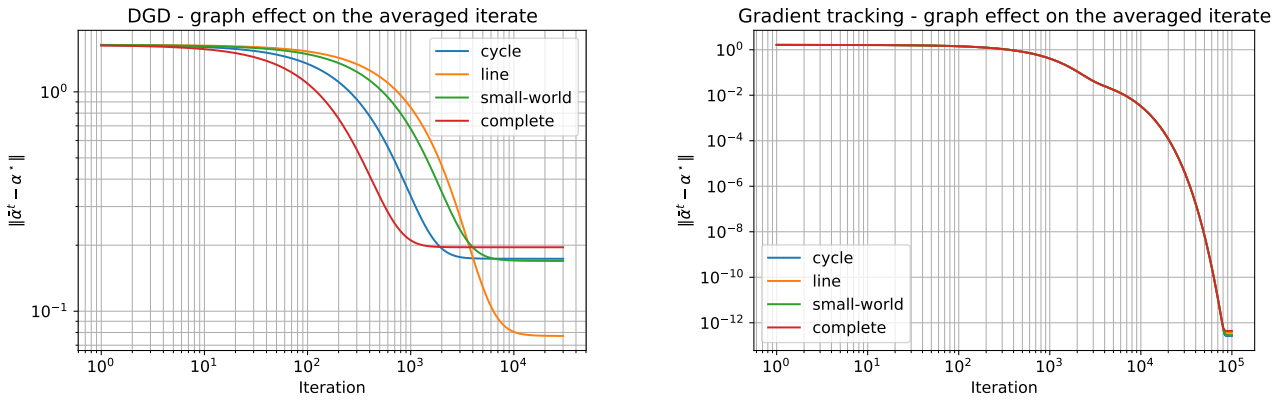

Figure 3: Influence of the graph topology on the averaged iterate for DGD (left) and GT (right).

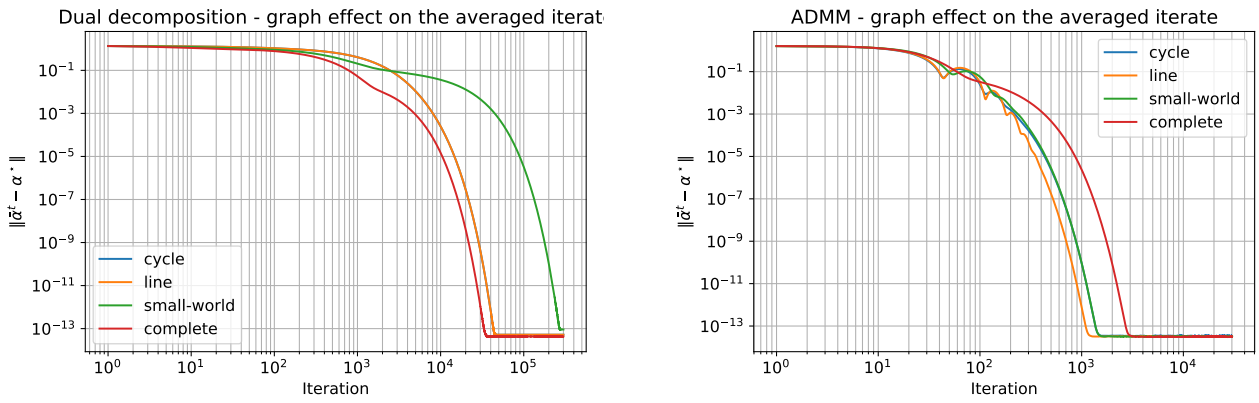

Figure 4: Influence of the graph topology on the averaged iterate for Dual decomposition (left) and ADMM (right).

Topology affects the methods differently. For DGD, better mixing mostly improves the transient phase and allows a larger admissible step; because $\eta_{\mathrm{DGD}} \propto (1-\beta)/L_{\max}$, the final bias is therefore not strictly monotone in β. For GT, the curves align much faster and all end at machine precision, confirming that the tracking term removes the steady-state bias observed in DGD. Dual decomposition and ADMM also reach the exact solution here because the consensus problem is quadratic, strongly convex, and solved through closed-form local updates.

We can also take a look at the function reconstruction for each algorithm.

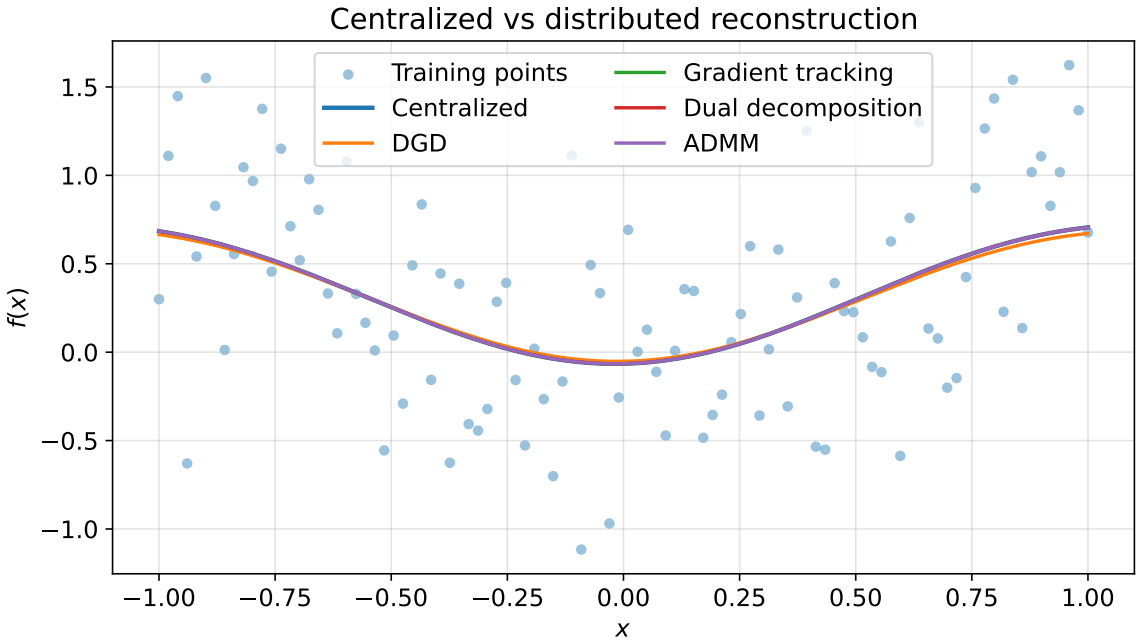

Figure 5: Reconstructed functions for the centralized solution and the four distributed methods.

We can see that for **DGD**, the curve is slightly offset from the others. This is consistent with its constant-step bias. By contrast, **GT**, **ADMM**, and **dual decomposition** are visually indistinguishable from the centralized reconstruction.

We can also perturb the communication model and study how this affects convergence: directed communication, packet losses, and asynchronicity. For now, if we make these changes, we obtain the following results for the DGD:

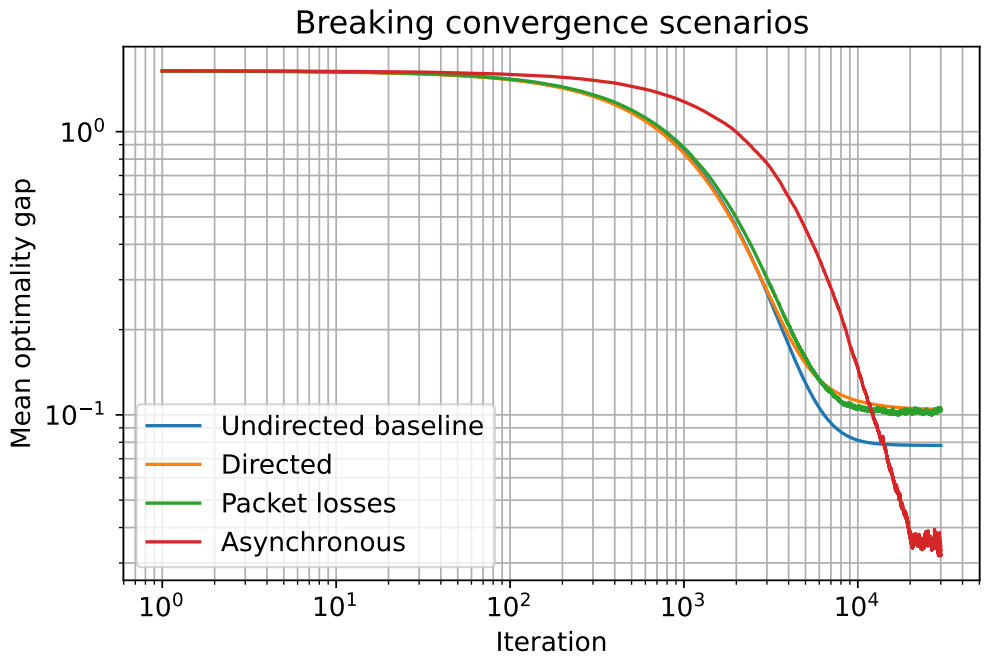

Figure 6: Breaking convergence assumptions for DGD: directed communication, packet losses, and asynchronous activations all degrade the nominal convergence behavior.

These perturbations break the standard DGD assumptions: directed communication destroys double stochasticity, packet losses create a time-varying possibly disconnected graph, and asynchrony violates the synchronous update model. The deterioration observed in Figure 6 is therefore consistent with the course theory.

Because of these modifications, we can use the push-sum protocol to recover convergence on directed graphs. We implemented it on top of DGD; the resulting error is lower and the method converges much more reliably than directed DGD alone.

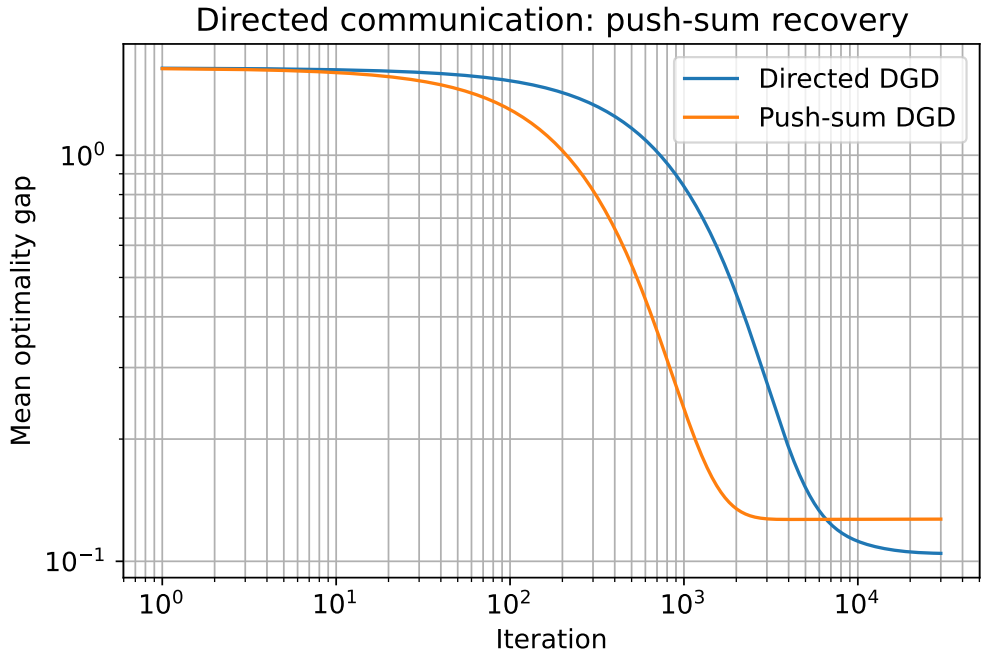

Figure 7: Directed communication: push-sum recovery compared with directed DGD.

Finally, we study the large-n regime. In order to maintain a valid Nyström approximation, we increase the model dimension m according to $m = \lceil \sqrt{n} \rceil$.

We implemented a dynamic search procedure to determine the system's maximum feasible scale. The algorithm iteratively increases n, adapts the number of agents when needed, and measures two key metrics:

- **Final accuracy**: The optimality gap reached after a fixed number of iterations.
- **Convergence rate**: The number of iterations required to fall below a critical threshold.

At the largest sizes, recomputing every Part I experiment against the exact centralized optimum is no longer practical. As allowed by the project statement, we then rely on surrogate metrics such as the consensus gap and the aggregate gradient norm, and we restrict the scalable sweep to the two first-order methods DGD and GT. Dual decomposition and ADMM are kept in the $n = 100$ reference study because their algebraic cost becomes prohibitive at the very largest scales.

Examining the generated figures (speed versus n) allows us to draw several conclusions:

1. **Computational Complexity**: Although the problem is distributed among a agents, an increase in n (and thus in m) increases the cost of each local operation.

2. **Consensus cost**: As n increases, the structure of the problem becomes more complex, and the number of iterations required to reach consensus increases.

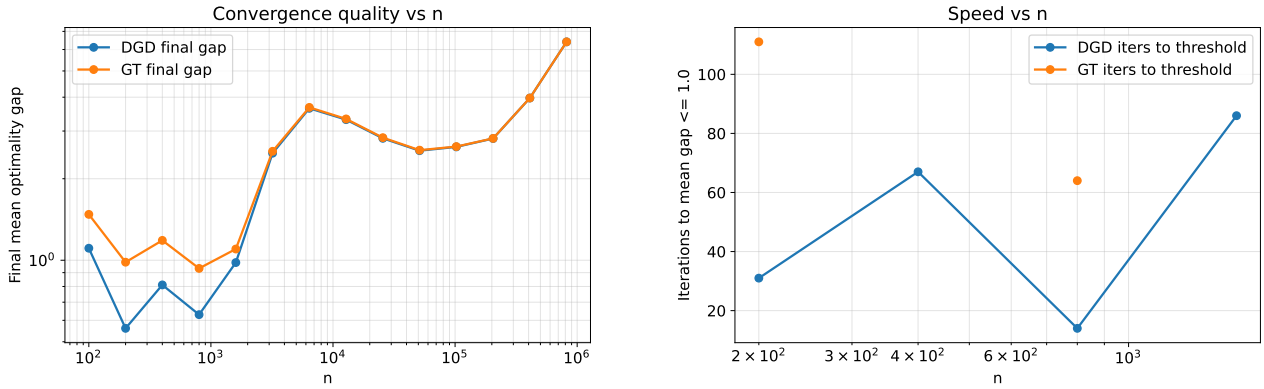

Figure 8: Large-n scaling: final mean optimality gap (left) and iterations needed to reach the target threshold (right).

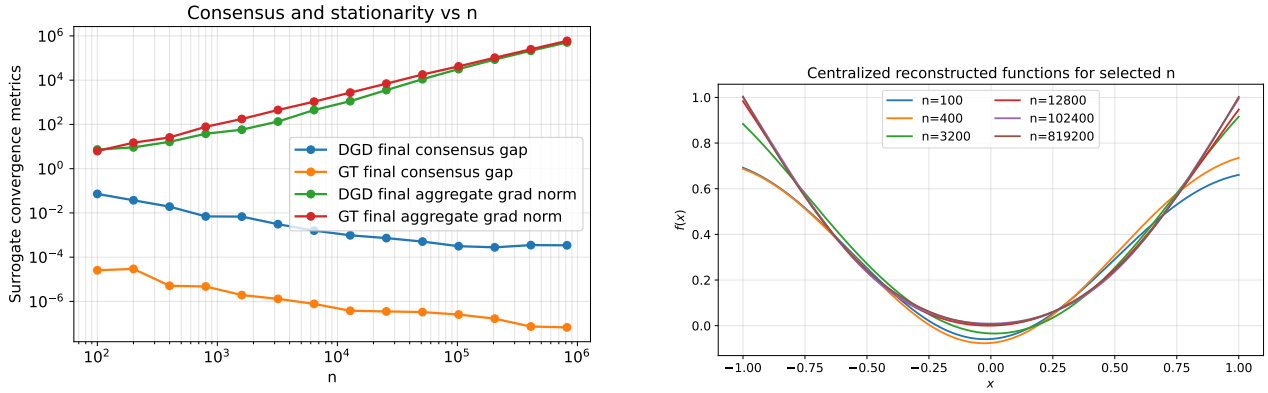

Figure 9: Large-n scaling: surrogate convergence metrics (left) and reconstructed functions for selected values of n (right).

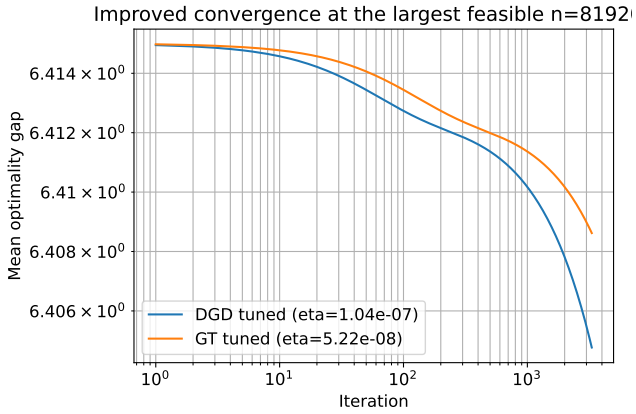

Figure 10: Retuned DGD and GT at the largest feasible size found under the current compute budget, $n = 819200$.

3 Federated learning

The second dataset contains five clients with 20 points each. The global objective keeps the same Nyström structure:

$$F(\alpha) = \sum_{c=1}^{5} \frac{1}{2} \|y_c - K_c\alpha\|^2 + \frac{\sigma^2}{2}\alpha^\top K_{mm}\alpha + \frac{\nu}{2}\|\alpha\|^2 .$$

The local mini-batch gradient used by FedAvg and SCAFFOLD is unbiased:

$$g_{c,b}(\alpha) = \frac{n_c}{|b|}K_{c,b}^\top(K_{c,b}\alpha - y_{c,b}) + \frac{\sigma^2}{5}K_{mm}\alpha + \frac{\nu}{5}\alpha.$$

The baseline local step size is

$$\eta_{\text{FedAvg}} = \frac{0.25}{L_{\max}^{\text{loc}}} = 3.136 \times 10^{-3}.$$

This keeps the local updates in a stable $O(1/L)$ regime. When $E = 50$, the step is halved to limit client drift, which naturally grows with the number of local epochs between two aggregations.

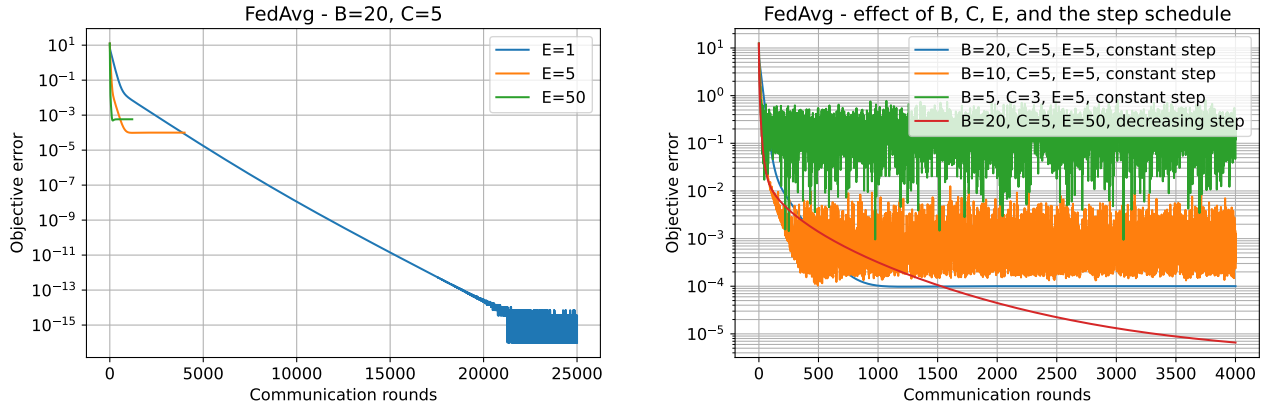

Figure 11: FedAvg with $B = 20, C = 5, E \in \{1, 5, 50\}$ (left), and the effect of B, C, E, and the step schedule (right).

The results display the usual trade-off. For $E = 50$, descent per communication round is fast, but it requires a more cautious step size. For $E = 1$, the trajectory is much slower in communication rounds,

yet it also reaches machine precision when the horizon is long enough. In the displayed runs, the final objective errors are $< 10^{-15}$ for $E = 1$, 1.00×10^{-4} for $E = 5$, and 5.81×10^{-4} for $E = 50$. The sensitivity plot confirms that decreasing B increases local variance, decreasing C slows down the global correction, and a decreasing step is mainly useful in the most aggressive settings.

The optional comparison with SCAFFOLD, run with $C = 3$ and $E = 5$, gives a final error 2.03×10^{-2} for FedAvg versus 2.13×10^{-14} for SCAFFOLD. This is consistent with the theory: the control variates compensate for client drift and become especially useful as soon as not all clients participate at every round.

4 Differential privacy

To solve the regression problem in a distributed manner while ensuring local data privacy, we implement the DGD-DP algorithm. Following the framework of **Theorem 2** from arXiv:2202.01113, the private update rule is defined as follows:

$$\alpha^{k+1} = \alpha^k + \gamma_k \big(W(\alpha^k + \zeta^k) - \alpha^k\big) - \eta_k g^k$$

where g_i^k is the local gradient and ζ_i^k represents the additive Laplace noise injected during the communication step.

To match the spirit of Theorem 2, part 3, we keep its main ingredients: Laplace perturbations, diminishing sequences γ_k and η_k, and the privacy levels $\epsilon \in \{0.1, 1, 10\}$. The theorem also requires a bounded sensitivity constant; in our implementation, this is instantiated by the proxy $C = 0.01$. This section should therefore be read as a theorem-guided study of the privacy/accuracy trade-off under the class assumptions.

Noise Mechanism and Sensitivity To satisfy ϵ-differential privacy requirements, the noise is drawn from a Laplace distribution $\zeta_i^k \sim \mathrm{Lap}(0, \nu)$, where the scale parameter ν is calibrated based on the local sensitivity C and the privacy budget ϵ:

$$\nu = \frac{C}{\epsilon}$$

In this implementation, we do not re-derive a data-dependent sensitivity bound; instead, we assume that the communicated quantity has bounded sensitivity with proxy $C = 0.01$. Under this assumption, the calibration $\nu = C/\epsilon$ is consistent with the Laplace mechanism used in the theorem-guided construction.

Convergence Conditions The persistent injection of noise necessitates the use of decaying step-size sequences to ensure stability and convergence toward the optimal solution α^*. We employ the following polynomial decay schedules:

$$\gamma_k = \frac{1}{1 + 0.001 \cdot k^{0.9}}, \qquad \eta_k = \frac{0.002}{1 + 0.001 \cdot k}$$

- $\epsilon = 10$: Low noise level; the scale ν is small, allowing for fast convergence that closely matches the deterministic DGD behavior.
- $\epsilon = 0.1$: High privacy level; the large noise scale ν induces significant variance in the consensus step. However, the decaying steps η_k and γ_k successfully dampen these oscillations, allowing the algorithm to stabilize in a neighborhood of the optimum.

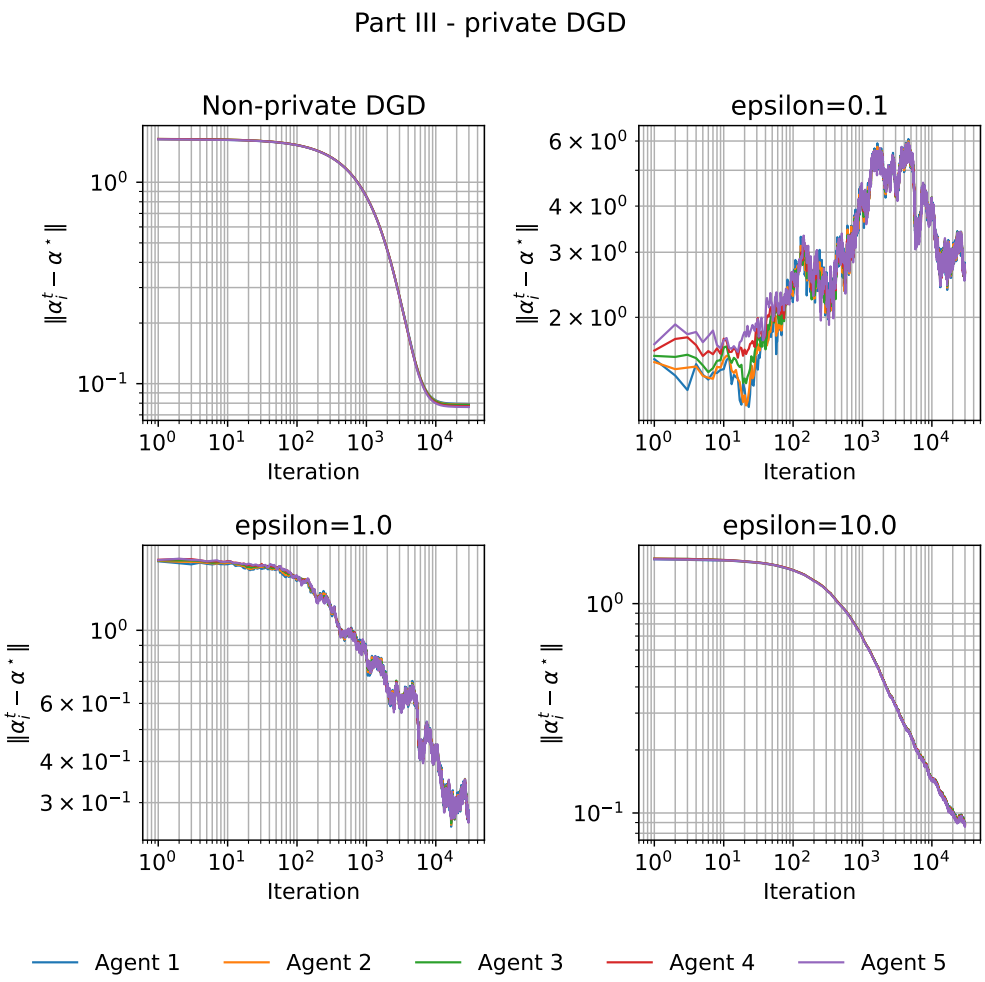

Figure 12: Agent-wise gaps for non-private DGD and private DGD with $\epsilon \in \{0.1, 1, 10\}$.

The analysis in Figure 12 highlights the fundamental trade-off in *Differential Privacy*.

- **Strong privacy** ($\epsilon = 0.1$): A high error plateau is observed. The injected Laplace noise dominates the gradient signal in the early iterations, preventing the algorithm from achieving high precision.
- **Moderate privacy** ($\epsilon = 10$): The curve converges to an error level close to that of classical DGD. The noise is sufficiently low to be absorbed by the gradient descent dynamics.
- **Effect of decreasing steps**: Unlike standard DGD, the use of decreasing steps η_t and γ_t is vital here. It helps stabilize noise-induced oscillations, although it slows down the overall convergence rate.

5 Conclusion

The experiments support the following conclusions:

- the Nyström kernel-ridge formulation is used consistently across the three parts, with the centralized solution as the reference point;
- in distributed optimization, DGD keeps a constant-step bias, whereas GT, dual decomposition, and ADMM converge exactly in this quadratic setting;
- in federated learning, increasing the number of local epochs speeds up progress per round but also increases client drift, and SCAFFOLD corrects that drift effectively;
- in differential privacy, DGD-DP remains stable but converges only to a neighborhood of the optimum, as expected with Laplace communication noise and decaying steps.

The numerical behavior is therefore coherent with the course notation and with the expected mecha-

nisms of the different algorithms.